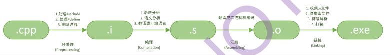
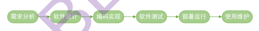
BASIS

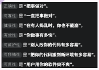
- chapter 1 程序 -> 软件
  - 程序
    - D：程序设计语言描述的一组能为计算机理解并处理的序列
    - 源代码：程序设计语言描述的代码
      可执行代码：可执行的二进制 or 中间码


.cpp
1.处理#include
2. 处理#define
3. 删除注释
预处理
(Preprocessing)
.i
1.语法分析
2. 语义分析
3. 翻译成汇编语言
编译
(Compilation)
.s
翻译成二进制机器码
汇编
(Assembling)
.o
1. 收集.o文件
2. 收集库文件
3. 符号解析
4. 打包
链接
(Linking)
.exe

    - 程序质量：
      外部质量(用户视角)：正确\可靠\方便实用\美观\高效 etc.
      内部质量(程序员视角)：可理解\可扩展\鲁棒\可重用\可维护\易修改 etc.
    - 保障质量的方法
      - 遵循编码风格
      - 采用程序设计方法：模块化\高内聚低耦合
      - 代码重用：反复调用而得到多次检验，可提升编程效率&质量
      - 结对编程
    - 质量检测分析方法
      - 人工代码审查
      - 自动化分析
      - 程序测试　利用测试用例运行测试
  - 软件
    - **程序 ＝ 算法+数据结构**
      软件 ＝ 程序+数据+文档
    - 软件生命周期


需求分析
软件设计
编码实现
软件测试
部署运行
使用维护

    - 特点
      - 逻辑性(不会物理老化) -> 回归测试
      - 设计开发(复制成本为0，重点在设计) -> 架构设计
      - 易变性(需求频繁变更) -> 敏捷开发
      - 复杂性(随模块数增加，复杂度指数增长) -> 模块化 & 解耦
      - 缺陷隐蔽性 -> 软件测试
    - 评价因素



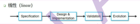
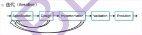
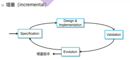
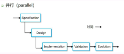
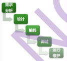
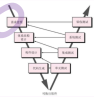
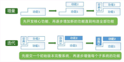
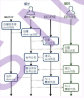
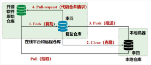
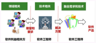
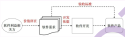
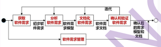
正确性 是 **“把事做对”。**
可靠性 是 **“一直把事做对”。**
健壮性 是 **“在有人捣乱时，你也不能崩”。**
有效性 是 **“你做事有多快”。**
可维护性 是 **“别人改你的代码有多容易”。**
可移植性 是 **“把你的代码搬到新环境有多容易”。**
可用性 是 **“用户用你的软件爽不爽”。**

    - 开源软件
      - 开源许可证:声规范开源软件的使用要求和约束
      - 二类许可证
        - 宽松式:限制少
        - Copyleft式：Copyright的反义词，允许自由使用、复制、修改、再分发；但要求派生作品也必须以相同的开放许可发布；
      - Apache开源许可证
        衍生品继续使用该许可证
        修改源代码需在文档中声明
        需保留原始代码内容信息
      - GPL开源许可证
        自由复制、传播
        可收费传播，但需告知可免费获取
        自由修改，但修改后仍使用该许可证
- chapter 2 SE BASIS
  - D：将系统的、规范的、可量化的方法应用于软件的开发、运行和维护的过程；以及上述方法的研究
    - 系统化 -> 解决流程混乱
    - 规范化：针对各类软件系统开发的各种标准 -> 解决协作混乱
    - 可量化：工作量、成本、进度、质量等要素 -> 解决管理混乱
  - 三要素
    - 过程
      - 回答 what & when
      - 是软件开发的生命周期框架，定义工作阶段\里程碑\顺序等
      - 目的：对项目的时间\成本\分工进行宏观管理和控制
      - 如瀑布模型\敏捷开发
    - 方法学
      - 回答 how
      - 实现过程中各阶段的技术指导思想和范式
      - 目的：为开发提供系统性的技术支持
      - 如结构化方法\OO
    - 工具
      - 回答 with what
      - 如 IDE\Git\SonarQube
  - 原则
    - 抽象&建模　抽象是"因"(思维活动)，建模是"果"(技术产出)
    - 分而治之(拆解复杂系统为一组子系统)
    - ❗ 模块化 -> 高内聚低耦合
      结果
    - 软件重用
    - ❗ 信息隐藏 (封装，提供接口)
    - ❗ 关注点分离　使开发者可以独立地、专注于某个特定方面（如：业务逻辑、用户界面、数据存储）进行开发和维护。
    - 双向追踪(软件发生变化，追踪影响&来源)
    - 工具辅助

要解决软件危机，首要任务是把软件活动视作可控的、可度量的和可改进的**过程**。

BLAIR'S NOTE

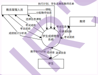
——Watts Humphrey

- chapter 3 软件过程模型和开发方法
  - 基本活动
    - 需求-》系统架构&实现 -》检验 -》变更
    - 组织形式
      - 
        线性（linear）
Specification
Design & Implementation
Validation
Evolution
      - 
        迭代（iterative）
Specification
Design
Implementation
Validation
Evolution
      - 
        增量（incremental）
Design & Implementation
Specification
Validation
Evolution
增量版本
      - 
        并行（parallel）
Specification
时间
Design
Implementation
Validation
Evolution
  - 软件过程模型
    - 传统/重量型过程模型
      - 瀑布模型 (Waterfall Model)：一种系统的、规范的、顺序的开发方法，适用于需求已确定且相对稳定的项目。
        
        需求分析
设计
编码
测试
运行维护
      - V模型 (V Model)：瀑布模型的变体，强调了测试活动与分析和设计的对应关系
        
        需求建模
验收测试
体系结构设计
系统测试
构件设计
集成测试
代码生成
单元测试
可执行软件
      - 原型模型 (Prototyping Model)：通过快速构建一个可运行的原型来明确模糊的需求。原型在获取用户真实需求后，可能会被抛弃，然后重新开始正式开发。
      - 增量模型 (Incremental Model)：先开发核心产品（第一个增量），然后在此基础上逐步交付一系列功能递增的版本（后续增量）。用户可以更早地使用核心功能。
        
        增量
功能1
功能2
功能3
先开发核心功能，再逐步增加新的功能直到构造全部功能
迭代
功能1
功能2
功能3
先提交一个初始版本完整系统，再逐步增强每个子系统的功能
      - 统一过程 (UP / RUP)：这是一种"用例驱动、以架构为核心、迭代且增量"的混合模型。它融合了传统模型的优点，并将开发分为四个阶段：Inception（初始）、Elaboration（精化）、Construction（构建）和Transition（交付）。
    - 敏捷/轻量型方法
      - 敏捷方法 (Agile Method)：为应对需求的快速变化而生，是一种以人为核心、迭代、增量的开发方法。它更强调"个体和交互"、"可运行的软件"、"客户合作"和"响应变化"。
      - 极限编程 (eXtreme Programming, XP)：敏捷方法的代表之一，强调四大核心思想（交流、反馈、简单、勇气），并包含一系列优秀实践，如测试驱动开发（TDD）和重构。
      - Scrum：一种敏捷的管理实践框架，定义了团队的三个核心角色（PO、Scrum Master、Team）和一系列管理活动（如每日站立会议、迭代计划、燃尽图）。
    - 群体化开发方法
      - 群体化开发 (Crowdsourcing Development)：源自开源软件的开发模式，依托互联网平台（如Github、Gitee），吸引和组织大规模的互联网大众（外围开发者）参与到软件开发中，兼顾软件的"创作"和"生产"。
        - 
          组长
组员
master
develop
自己分支
自己分支
创建版本库 push代码
创建 develop分支
创建 自己分支
创建 自己分支
合并分支
提交、push代码
解决冲突 合并分支
提交、push代码
合并 develop
Pull 最新代码
Pull 最新代码
        - 基于P/R的分布式协同开发
          
          开源软件原始仓库
4. Pull-request（代码合并请求）
李四
1. Fork（复刻）
3. Push（推送）
复刻仓库
本地机器
在线平台和远程仓库
2. Clone（克隆）
李四
Pull（拉取）
本地仓库
- 软件需求和需求工程
  - 软件需求(Software Requirement, 缩写为SR):
    - 软件本身角度：软件用于解决现实世界问题所提出的功能\性能等方面的要求
    - 软件利益相关方角度：利益相关方对软件系统的功能\质量\运行环境\交付进度等的期望和要求
    - 
      领域相关
技术相关
融合需求和技术
期望和要求
解决方案
软件产品
软件利益相关方
软件工程师
软件工程师
    - 利益相关方：从软件系统中受益或相关的人\组织\系统
      - 用户(user)\客户(从中获利的组织)\系统(与该系统交互的系统，如部署该系统的机器人)\开发者
    - 类别:
      - 功能需求Functionality：能完成的功能及具体场景下外部可见行为和效果
      - 质量需求Quality：外部质量属性(用户\客户关心的性能\易用性等)，内部质量属性(开发者关心的可扩展性\可维护性等)
      - 开发约束需求Constraint：开发成本\交付进度等
    - 重要性(作用之处)
      
      验收标准
软件利益相关方
价值所在
软件需求
开发依据
软件开发
软件产品
  - 需求工程：对系统服务和约束进行理解\分析\建立文档\检验的过程
    - 
      迭代
获取软件需求
初步软件需求
分析软件需求
软件需求模型
文档化软件需求
软件需求文档
确认和验证软件需求
确认后的需求模型和文档
软件需求管理
    - 方法学：抽象\建模\分析

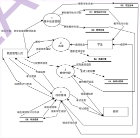
BLAIR'S NOTE

BLAIR'S NOTE

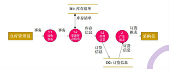
- chapter 4 SR工程基础
  - 结构化需求分析方法学
    - 结构化分析：考虑数据和处理，面向数据流，基本思想为分解和抽象
    - DFD图(数据流图, Data Flow Diagram)
      - 顶层


教务管理人员
执行计划、学生名单和教师名单
课程
分配教师信息
选课信息课程
考试成绩
考试成绩
成绩统计分析表
学生成绩管理系统
教师
考试信息
任课通知单
考试成绩
教学执行计划
成绩单
选课信息
学生

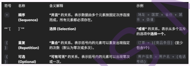
      - 1层


更新学生名单
D3：学生名单
更新教学执行计划
D1：教学执行计划
1 基本信息管理
更新教师名单
D2：教师名单
教学执行计划
执行计划、学生名单和教师名单
课程
2 选课
选课信息
学生
成绩单
选课信息课程
教务管理人员
更新选课记录
D4：选课记录
分配教师信息
读取选课记录
3 教师分配
生成分配结果
D5：教师分配表
考试成绩
考试成绩
查询教师信息
成绩统计分析表
4 成绩管理
任课通知单
考试信息
考试成绩
教师
输出成绩单
输出成绩统计分析表
更新学生成绩
D6：考试成绩
输出学生信息

      - 2层(例)

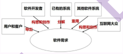

D1: 库存清单
库存清单
仓库管理员
事务
1.1 接收事务
事务
1.2 更新库存清单
库存信息
1.3 处理订货
2 产生报表
订货报表
采购员
订货信息
订货信息
D2: 订货信息

    - 数据字典(DD)
      - DD存储元数据，是all与系统相关的数据元素的有组织的列表，有精确\严格的定义

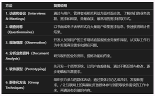


符号	名称	含义解释	示例
+	顺序 (Sequence)	"并且" 的关系。表示数据由多个元素按固定次序连接而成，所有元素都必须存在。	`地址 = 国家 + 省份 + 城市 + 街道`
**`[	]`**	选择 (Selection)	"或者" 的关系。表示从多个互斥的选项中选择一个。
{ }	重复 (Repetition)	"集合" 的关系。表示括号内的元素可以重复出现指定的次数（默认为零次或多次）。	`订单 = 1{商品条目}` (至少包含1个)
( )	可选 (Optional)	"可有可无" 的关系。表示括号内的元素可以出现零次或一次。	`用户信息 = 用户名 + (电话号码)`

      - 基本表示符合
        - 支付方式 = [信用卡 | 支付宝 | 微信支付 | 银行转账]
        - m{a}n：表示元素 a 至少重复 m 次，最多重复 n 次。
    - 缺点：面向功能，而功能因素并不稳定，数据与操作分离
  - 面向对象需求分析方法学
    - OO=objects+class+inheritance+communication
    - 面向对象建模语言
      - UML(Unified Modeling Language)
        - 结构视角(系统构成)：包图\类图\对象图\构件图
        - 行为视角：交互图\状态图\活动图
        - 部署视角(系统在物理层面的结构与运行环境)：部署图
          - [客户端浏览器] ⇄ [Web服务器] ⇄ [数据库服务器]
        - 用例视角(功能)：用例图
- chapter 5 获取SR

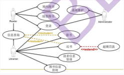

软件开发者
已有的系统
其他软件系统
用户和客户
构思和创作
分解
重用
互联网大众
导出
构思和创作
软件需求

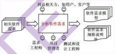
  - 获取SR的方法
    - 访谈和会议
    - 调查问卷
    - 现场观摩
    - 分析业务资料
    - 软件原型
    - 群体化方法

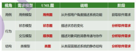


方法	简要说明
1. 访谈和会议（Interviews & Meetings）	通过与用户、管理者或相关利益方面对面交流，了解他们的业务流程、需求和期望，是最直接、最常用的需求获取方式。
2. 调查问卷（Questionnaires）	以书面或电子表单形式向大量用户收集需求信息，快速获得统计性结果。
3. 现场观摩（Observation）	开发人员到用户的工作现场直接观察业务操作流程，从实际工作行为中发现真实需求和潜在问题。
4. 分析业务资料（Document Analysis）	研究现有的业务资料，提炼功能和约束。
5. 软件原型（Prototyping）	先开发一个原型系统，让用户直观体验，通过不断反馈与修改，逐步明确和完善需求。
6. 群体化方法（Group Techniques）	组织多方参与的群体活动，通过集体讨论达成共识、发现新需求。/ 让互联网上的海量和开放群体参与到获取软件需求的工作中来，再遴选有价值的内容。

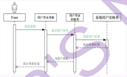
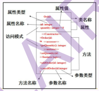
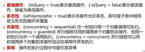
  - 明确问题及S解决方案
    - 定义问题：调研分析(切忌凭空想)\反复论证\要有意义有价值
    - 明确基于<软件>的解决方案
  - 导出和构思SR
    - SR来自利益相关方，so 需要首先进行识别
    - 功能性R　站在不同利益相关方的视角去分析SR
    - 非功能性R　质量/约束性R
  - 描述初步的SR
    - 自然语言描述　表达能力强\直观\普适，but存在模糊性\二义性
    - 用例图
      - 元素：系统(提供用例的黑盒)\行为者\用例\用例间关系
      - 关系
        - 包含(include)：B是A的子功能，则A包含B　包含用例 -> 被包含用例
        - 扩展(extend)：插入新行为　扩展用例 -> 被扩展用例
        - 泛化（Generalization）　子用例 -> 父用例
        - 总结：依赖 -> 被依赖


预约图书
续借图书
系统维护
Reader
登录
Administrator
信息查询
<<include>>
借书
<<include>>
还书
<<extend>>
超期罚款
Librarian
读者信息管理
图书信息管理

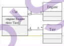
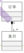
- 需求
- chapter 6 分析SR
  - 概述
    
    利益相关方，如用户、客户等
初步软件需求
分析软件需求
软件需求模型
软件需求规格说明书
需求工程师
项目管理者
测试和设计工程师

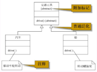
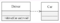
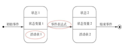
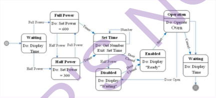
    

视角	需求模型	UML图	说明	阶段
用例	用例模型	用例图	从外部用户角度描述系统功能	获取软件需求
行为	状态模型	状态图	描述状态的变化	分析软件需求
	交互模型	顺序图	描述对象间的消息传递与协作	分析软件需求
结构	类模型	类图	从类层面描述系统的静态结构	分析软件需求

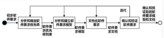
  - UML图
    - 顺序图：系统中对象间的交互建模，是对用例图中用例交互顺序的补充说明
      
      User
用户登录界面
用户登录控制类
系统用户实体类
登录系统
验证用户身份
查找用户记录
返回查询结果
为用户授权
显示登录信息
    - 类图：描述系统的类构成，刻画静态组成结构
      
      属性类型
属性值
属性名称
类名称
属性
访问模式
方法
参数类型
方法名称
参数名称
Order
-id: integer
-quantity: integer = 0
<<Constructor>>
+Order(id: in
<<accessor>>
+getQuantity(): integer
<<mutator>>
+setQuantity(id:
<<business logic>>
+addOrder(id: integer)
+cancelOrder(id:integer
)
      - 属性
        - 格式：[可见性] 名称 [: 类型] [多重性] [= 初值] [{约束特性}]
        - e.g. - id: String [1] = "unknown" {readOnly, static, ordered}
        - +公开 #保护 -私有
        - 多重性：属性取值数量, 如1，0..1，0..*，1..*，*
      - 方法
        - 格式：[可见性] 名称 [(参数表)] [: 返回类型] [{约束特性}]
        - e.g. + withdraw(amount: Double) : Boolean {exception=InsufficientBalance}
        - 约束特性
          

          - **查询操作**：{isQuery = true}表示查询操作，{ isQuery = false}表示修改操作，缺省为修改操作。
          - **多态性**：{isPolymorphic = true}表示本操作允许多态，即可被子类中相同定义形式的操作所覆盖
          - **并发性**：{concurrency = sequential} 任一时刻只有一个对象调用可执行。{concurrency = guarded} 并行线程可同时调用多个对象的本操作，但同一时刻只允许一个调用执行。{concurrency = concurrent} 并行线程可以同时调用多个对象的本操作且这些调用可并发执行
          - **异常**：操作在执行过程中可能引发异常

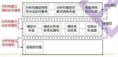
          - 对一个元素（比如属性或操作）增加额外限制或行为说明的"说明性附加条件"。
      - 接口
        - 供给接口：提供某种服务
          - [类或组件]——○
          - e.g. BankService ——○ IAccountService
        - 需求接口：需要某种服务
          - [类或组件]——(
          - e.g. ATM ——(IAccountService
      - 关系
        - 关联
          
          Car
-engine:Engine
-tires:Tire[]
Engine
Tire
1
1
1
0..*
          - 聚合
          

          - 组合
        - 泛化(继承)
          
          交通工具
{abstract}
附加标记
drive() {abstract}
普通泛化
汽车
drive( )
船
drive( )
驱动车轮转动
注释
转动螺旋桨
        - 依赖
          
          Driver
+drive(Car car):void
Car
    - 状态图：描述实体在事件作用下的状态变化
      
      初始事件
状态 1
状态变量 1
活动表 1
事件表达式
状态 2
状态变量 2
活动表 2
结束事件
      - 事件表达式格式：事件说明[守卫条件]: 动作表达式
      - 活动表格式：事件名(参数表):动作表达式
        - 常用事件：entry\exit\do
      - e.g.
        
        Waiting
Do: Display Time
Full Power
Full Power
Do: Set Power = 600
Half Power
Full Power
Half Power
Half Power
Do: Set Power = 300
Timer
Timer
Set Time
Do: Get Number
Exit: Set Time
Number
Half Power
Disabled
Do: Display "Waiting"
Door Closed
Door Closed
Enabled
Do: Display "Ready"
Start
Operation
Do: Operate Oven
Cancel
Waiting
Do: Display Time
Door Open

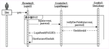
BLAIR'S NOTE


初步软件需求
分析和确定软件需求优先级
软件需求优先级列表
分析和建立软件需求模型
软件需求模型
文档化软件需求
软件需求文档
确认和验证软件需求
迭代
确认和验证后的软件需求模型和文档

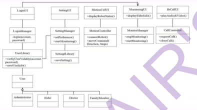
- 过程
  - 需求优先级
    - 重要性：核心\外围
    - 优先级：高中低
    - 迭代次序：优先级高的先进行迭代实现
  - 建模

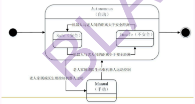

分析和建立用例交互模型
分析和确定用例所涉及的对象类
分析和确定对象间消息传递
绘制用例的交互图
交互模型
分析和建立分析类模型
确定分析类
确定分析类职责和属性
确定类间关系
绘制分析类图
类模型
分析和建立状态模型
绘制状态图

    - 用例交互模型
      - 用例涉及的对象及其类
        - 边界类
        - 控制类
        - 实体类
      - 消息传递：交互信息
      - 顺序图


User
LoginUI
LoginManager
UserLibrary
login(account, password)
verifyUserValidity(account, password)
LoginResult(FAILED)

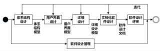
        - 顺序图：交互图最常用的一种。强调对象之间消息传递的时间顺序。
    - 分析类模型
      - 确定分析类及其职责、属性、关系 -> 绘制分析类图


LoginUI
SettingUI
MotionCtrlUI
+displayRobotStatus()
MonitoringUI
+displayElderInfo()
BiCallUI
+playAudio&Video()
LoginManager
-login(account, password)
SettingManager
+setPreference()
+startMonitoring()
MotionController
+connectRobot()
+moveCommand(Direction, Steps)
MonitorManager
+stopMonitoring()
+startMonitoring()
CallController
+requestCall()
+closeCall()
UserLibrary
+verifiyUserValidity(account, password)
+saveUserInfo()
SettingfLibrary
+saveSetting()
User
Administrator
Elder
Doctor
FamilyMember

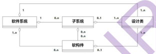

Autonomous
（自动）
机器人与老人间的距离大于安全距离
Safe（安全）
Unsafe（不安全）
机器人与老人间的距离少于安全距离
老人家属或医生结束机器人运动控制
老人家属或医生要控制机器人运动
Manual
（手动）

    - 状态模型
      - 状态是针对对象的，不是针对分析类
- 文档化及评审

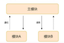
- BASIS
  - 软件设计：针对SR，综合考虑制约因素，给出解决方案


体系结构设计
体系结构模型
用户界面设计
用户界面模型
详细设计
详细设计模型
文档化软件设计
软件设计文档
软件设计评审
迭代
软件设计管理

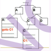
  - 设计过程
    - 体系结构设计：从全局和宏观视角、立站在最高抽象层次来设计软件系统
    - 用户界面设计：设计软件对外展示以及与用户进行交互的界面
    - 详细设计
      - 精细化，获得高质量的、充分细化的软件设计模型
      - 数据、接口、类、算法设计


软件系统
子系统
设计类
软构件
1
1
0..n
0..1
1..n
1..n
1..n
1..n
0..n
0..n
0..n
0..1

    - 软件设计元素
      - 软构建：可分离、可独立部署和执行、可单独重用的一类设计元素
      - 连接线两端数字：首端表指向，尾端表接收
        - 如软件系统与子系统，一个子系统只能指向一个软件系统，一个软件系统可以接收0..n个子系统
- S设计原则
  - 1. 抽象与逐步求精
  - 2. 模块化
  - 3. 高内聚和低耦合：模块内高内聚，模块间低耦合
    - 耦合分类


主模块
通知
调用
模块A
模块B

      - 非直接耦合：两模块无直接联系，通过主模块间接实现
      - 数据耦合：两模块间仅传递数据项参数
      - 控制耦合：传递控制信息
      - 特征耦合：传整个数据结构（记录、数组等）
      - 外部耦合：依赖于由外部环境强加的特定格式、通信协议或硬件接口
      - 公共耦合：一组模块都访问同一个全局数据结构


A
B
C
D
A:
goto C1
C:
C1:

      - 内容耦合：一个模块直接转移到另一模块内执行（需避免）

软件工程 SE

BLAIR'S NOTE

转换视角
你是一名软件工程师
不是一名程序员

图例

- ⭐ Star
- ❗ 2025年考察知识点

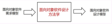
❗ 4. 信息隐藏：信息只展示或交换给需要的模块（private、public等）

5. 软件重用：封装、接口、继承、多态等

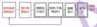
chapter 7 S设计基础

面向对象S设计方法学

面向对象需求分析得到SR模型(用例图\交互图\分析类图etc)
-> 精化得到各类软件设计软塑(子系统\构建\设计类etc)
-> 软件设计模型(S体系结构图\用例设计交互图\设计类图\活动图etc)
-> 完整详尽的设计方案


面向对象软件需求模型
面向对象软件设计方法学
面向对象软件设计模型

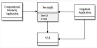
面向对象S设计过程

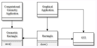

软件体系结构设计
用户界面设计
用例设计计设计
构件/子系统设计
数据设计
类设计
详细设计

设计原则

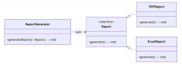
单一职责原则：一个类只负责一项职责

e.g. 拆分


Computational Geometry Application
Rectangle
draw()
area()
Graphical Application
GUI


Computational Geometry Application
Graphical Application
Geometric Rectangle
area()
Rectangle
draw()
GUI

开闭原则：对扩展开放，对修改关闭


ReportGenerator
+generateReport(r: Report) : : void
uses
«interface»
Report
+generate() : : void
PDFReport
+generate() : : void
ExcelReport
+generate() : : void

里氏替换原则：子类对象必须能够替换掉父类对象，并且程序的行为不发生变化

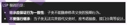
e.g.

```
class Bird {
    public void fly() {
        System.out.println("I can fly");
    }
}

class Ostrich extends Bird {
    @Override
    public void fly() {
        throw new UnsupportedOperationException("Ostrich can't fly!");
    }
}
```

```
Bird bird = new Ostrich();
bird.fly(); // ❌ 程序异常：鸵鸟不会飞
```

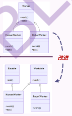
解决方法：不要让 Ostrich 继承 Bird，而是提取一个新的抽象

```
interface Bird {}
interface Flyable {
    void fly();
}

class Sparrow implements Bird, Flyable {
    public void fly() { System.out.println("I can fly"); }
}

class Ostrich implements Bird {
    // 不实现 Flyable
}
```


LSP 强调的是：

- **继承应保证行为一致性**：子类不能随意修改父类的预期行为。
- **不要强行继承**：当子类无法完美替代父类时，应考虑抽象、接口分离等设计。

接口隔离原则


Worker
+work()
+eat()
HumanWorker
+work()
+eat()
RobotWorker
+work()
+eat()
改进
Eatable
+eat()
Workable
+work()
HumanWorker
+work()
+eat()
RobotWorker
+work()

依赖倒置原则：高层模块不依赖低层，而应该依赖抽象；抽象不依赖具体实现，具体实现依赖抽象.

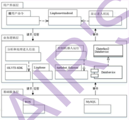
最少知识原则(迪米特法则)：一个对象对其他对象了解尽可能少

S设计输出　软件设计模型\文档

S体系结构设计（S架构）：从高层抽象角度刻画设计元素及其关联

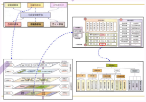
设计元素

- 构件（component）：物理模块，有特定功能和对外接口
- 连接件（connector）：构件间的联系和交互
- 约束（constraint）


用户界面层
接用户命令
Linphone4Android
显示老人状况
请求/应答
事件
业务逻辑层
分析和处理老人信息
控制机器人运行
《Interface》DataService
OLVTI-SDK
Linphone
turtlebot_follower
DataService
请求/应答
事件
基础服务层
ROS
MySQL

Krutchen的4个体系结构视图

- 逻辑视图
  - 包图
  - 构件图
- 开发视图：项目目录结构
- 物理视图：部署图
- 运行视图：活动图、对象图


②制造模块
③建筑模块
④电商模块
①企业互联平台
⑤培训模块
⑥服务模块
⑦××模块

BASIS　体系结构描述

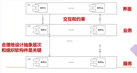
BLAIR'S NOTE

BLAIR'S NOTE

设计

chapter 8 S体系结构设计

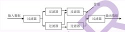
体系结构风格 Style


界面
交互和约束
业务
合理地设计抽象层次
和组织软构件是关键
服务
……

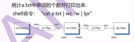
分层风格：按抽象级别组织为若干层

管道/过滤器风格

- 构件：将每个处理步骤封装在一个过滤器构件中
- 连接件：相邻过滤器以管道连接


输入数据
过滤器
管道
输出数据

Shell命令实例

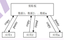


统计a.txt中单词的个数并打印出来：

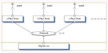
shell命令： "cat a.txt | wc -w | lpr"

cat Source
p1 Pipe
wc Filter
p2 Pipe
lpr Sink

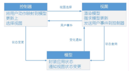
信息库风格：各系统构件相互独立、不交互，但共同访问同一中心数据存储中心


黑板
copy
paste
应用1
应用2
应用n

Client/Server：分布式体系结构，系统被呈现为一组服务， 每项服务由一个独立服务器提供

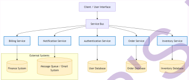

user1
user2
user3
a:FtpClient
b:FtpClient
c:FtpClient
Network
FtpServer

MVC风格

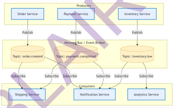

控制器
将用户动作映射到模型
更新上
选择视图
视图选择
用户事件
视图
渲染模型
强求模型更新
发送用户事件到控制器
状态变更
变化通知
状态查询
模型
封装应用状态
通知视图状态变更

SOA风格

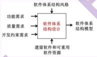
Service-Oriented Architecture，通过将系统功能封装为可独立访问、可复用、可组合的服务（Service），实现不同系统之间的松耦合集成与灵活协作。

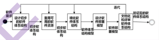

Client / User Interface
Service Bus
Billing Service
Notification Service
Authentication Service
Order Service
Inventory Service
External Systems
Finance System
Message Queue / Email System
User Database
Order Database
Inventory Database

消息总线风格

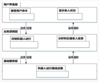

Producers
Order Service
Payment Service
Inventory Service
Publish
Publish
Publish
Message Bus / Event Broker
Topic: order.created
Topic: payment.completed
Topic: inventory.low
Subscribe
Subscribe
Subscribe
Subscribe
Subscribe
Subscribe
Consumers
Shipping Service
Notification Service
Analytics Service

S体系结构设计

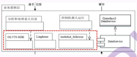

软件体系结构风格
功能需求
质量需求
开发约束需求
软件体系结构设计
软件体系结构模型
遗留软件和可重用软件资源


软件需求
设计初步的软件体系结构
初步软件体系结构
重用可用的软件资源
初步软件体系结构
精化软件体系结构
软件体系结构模型
设计软件部署模型
软件部署模型
文档化和评审软件体系结构
验证后的软件体系结构
迭代

设计初步S体系结构：根据SR & 业界已有Style进行设计，明确各构件职责&关系，用包图表示

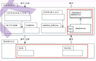

用户界面层
接受用户命令
显示老人状况
业务逻辑层
请求/按键
事件通知
控制机器人运行
分析和处理老人信息
基础服务层
机器人运行基础设施

重用开源软件及已有软件资产

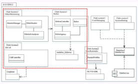

业务逻辑层
分析和处理老人信息
控制机器人运行
OLVTI-SDK
Limphone
turtlebot_follower
DataService

精化软件体系结构

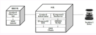
公共基础设施：基础性服务(如隐私保护服务、安全控制服务)



设计元素：子系统\软构件\关键设计类，明确职责&接口



设计部署模型：各子系统\软构建如何部署到计算节点上运行，描述其部署&运行环境

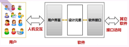


文档化及评审

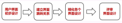
输出

- 软件模型：UML包图\部署图etc
- 软件文档：体系结构设计规格说明书文档

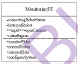
- chapter 9 用户界面设计
  - BASIS

    
    用户
人机交互
用户界面
设计元素
软件接口
软件
其它软件
接口访问

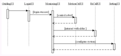
    - 人机交互常见方式：文本（命令行）、图形化界面、语音、手势（gesture）
    - 用户界面元素
      - 静态元素（与运行状态无关，始终不变，如图片、logo）
      - 动态元素（随系统运行状态或数据变化而自动更新的显示元素，如状态栏、进度条、实时统计表）
      - 用户输入元素（如输入框）
      - 用户命令元素（"触发动作"型控件）
  - 设计
    - 设计原则
      - 简洁直观
      - 便于使用，简单
      - 反应迅速
      - 一致性（风格统一&业界规范）
      - 容错性
      - 人性化
    - 设计过程

      
      用户界面初步设计
建立界面跳转关系
精化各个界面设计
评审界面设计

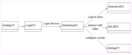
      - 界面init
        - 需求-》界面：用例模型&用户交互模型
          设计四类元素

          
          MonitoringUI
-connectingRobotStatus
-batteryofRobot
<<input>>-openCamera
-videoRegion
+monitorElder
+controlRobot
+interactElder
+configureSystem

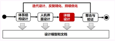
      - 建立跳转
        - 顺序图

          
          GuidingUI
LoginUI
MonitoringUI
MotionCtrlUI
ItsCallUI
SettingUI
[login success]
[control robot]
[interact with elder]
[configure system]

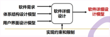
          交互图：特定跳转及其消息传递
        - 类图：跳转及其原因

          
          MotionCtrlUI
GuidingUI
LoginUI
Login Success
MonitoringUI
control robot
interact with elder
ItsCallUI
configure system
SettingUI

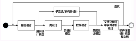
- BASIS

  
  迭代设计、反复精化、持续优化
体系结构设计
人机界面设计
详细设计
整合与验证
设计模型和文档

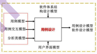
  - 已得到：S体系结构设计、用户界面设计、SR
    详细设计目标：细化、精化 -》高质量、面向实现（直接支持程序设计&编程）的设计模型

    
    软件需求
体系结构设计模型
用户界面设计模型
软件详细设计
软件详细设计模型
实现约束和限制

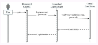
  - 软件详细设计活动

    
    迭代
子系统/软构件设计
用例设计
用例设计模型
类设计
类设计模型
数据设计
数据设计模型
文档化和评审软件详细设计
软件详细设计模型和文档

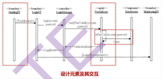
    - 用例设计：给出用例的具体实现（各设计元素：子系统、软构件、设计类等，之间的交互协作）

      
      软件体系结构设计模型
用例模型
用例交互模型
分析类模型
用例设计
用例设计模型
软件设计模型
用户界面模型

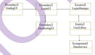
      - 例：需求分析阶段的用例顺序图

        
        User
《boundary》LoginUI
《controller》LoginManager
《entity》UserLibrary
login(account, password)
verifyUserValidity(account, password)
UserValidity
LoginResult

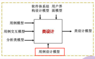
        - 在分析阶段用例顺序图的基础上，将顺序图的分析类(即边界类\控制类\实体类)转化为用例实现的设计类(转化后，程序员可以直接照着编程的类)
      - 例："用户登录"用例设计方案
        - 用例设计顺序图

          
          <<boundary>> GuidingUI
<<boundary>> LoginUI
<<controller>> LoginManager
<<entity>> UserLibrary
<<Component>> DataService
<<boundary>> MonitoringUI
[Touch]redirect to
login(account, password)
verifyUserValidity(account, password)
isUserValid(account, password)
Query
VerificationResult
LoginResult
[LoginSuccess]redirect to
设计元素及其交互

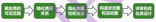
        - 设计类图

          
          《boundary》GuidingUI
《boundary》LoginUI
《control》LoginManager
《boundary》MonitoringUI
《entity》UserLibray
《component》DataService

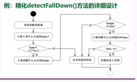
    - 类设计：属性定义&方法实现

      
      软件体系结构设计模型
用户界面模型
用例模型
用例交互模型
分析类模型
类设计
类设计模型
用例设计模型

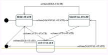
      - 精化类，下一步就可以直接根据类设计模型写代码

      
      确定类的可见范围
精化类间关系
精化类属性和方法
构造状态图和活动图
评审和优化类设计

      - 1. 类的可见范围：public/protected/private
      - 2. 类间关系：继承\组合\聚合\关联\依赖
      - 3. 类的属性和方法(活动图描述)

        
        例：精化detectFallDown()方法的详细设计
获取传感器数据
计算人体中心点速度SpineV
SpineV >1.37m/s?
是
否
计算两髋中心点高度Height
Height <0.22m?
计算两髋中心点停留时间Time
Time>5s?
没有检测到摔倒
检测到老人摔倒

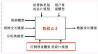
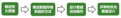
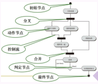
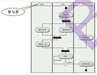
        输出类图
      - 4. 对象的状态图(可选)

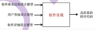
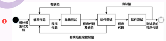
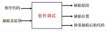
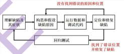
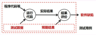
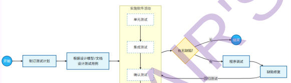
        
        setState(IDLE-STATE)
IDLE-STATE
setState(MANUAL-STATE)
MANUAL-STATE
setState(AUTO-STATE)
setState(AUTO-STATE)
setState(IDLE-STATE)
AUTO-STATE
setState(MANUAL-STATE)

      - 5. 评审优化："高内聚\低耦合"，简单\自然，编程实现便利度，信息隐藏

BLAIR'S NOTE

BLAIR'S NOTE

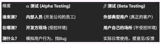
- chapter 10 S详细设计
  - 
    软件体系结构设计模型
用户界面模型
用例模型
用例交互模型
分析类模型
数据设计
数据设计模型
用例设计模型 类设计模型
  - 数据设计：数据及其操作
    - 
      确定持久数据
确定数据存储和组织方式
设计数据访问操作
评审和优化数据设计
    - 确定哪些数据需要永久保存，保存在数据文件or数据库中
      - 确定持久数据条目
        - 类->表格，对象->记录，属性->字段
        - 1:n关联使用外键连接
          m:n关联引入交叉新表纳入两外键
  - 子系统/软构件设计：给出细粒度设计元素(子构件/类)
  - 活动图
    - 
      初始节点
分叉
动作节点
控制流
合并
判定节点
最终节点
  - 泳道图
    - 

  - 文档化&评审
    - 输出
      - 模型：UML类图\构件图\包图\状态图\顺序图\活动图etc
      - 文档：软件详细设计规格说明书
    - 验证整个设计的正确性\优化性\充分性，模型间不一致性\冗余性，并整改
- chapter 11 BASIS
  - 
    软件体系结构设计模型
用户界面设计模型
软件详细设计模型
软件实现
高质量的程序代码
  - 
    有缺陷
有缺陷
设计模型和文档
编写代码
程序代码
单元测试
程序代码及缺陷
软件测试
程序代码
软件测试
测试后的程序代码
有缺陷且定位缺陷
  - 软件实现：编码、测试、调试etc
  - 程序设计语言的选择：考虑应用领域\与遗留软件系统的交互\特殊需求\目标平台\熟悉程度
  - 编码原则：易读\易改\复杂度低\可重用\异常处理&容错\与文档一致
- chapter 12 编写代码
  - 编写
    - 编写类代码
      - 类的代码(基于设计类图)
        类方法的代码(基于活动图)
        类间关联的代码(考虑语义信息)
        设计类间聚合和组合关系的代码
        接口关系的代码(基于类设计模型中的表征类&接口之间实现关系的语义信息)
        继承关系的程序代码
        包的代码
    - 编写用户界面代码
    - 编写数据设计代码(数据库中数据存储形式及其操作)
  - 调试
    - 软件缺陷
      - 软件缺陷带来的问题：错误(状态结果) & 失效(功能行为)
      - 状态：尚未确认\有效\无效\重复->已分配\已修复\描述信息不完整\已解决\已关闭(不再处理该缺陷)
    - 
      程序代码
缺陷及症状
软件调试
缺陷原因
缺陷位置
修复缺陷后的代码
    - 步骤
      - 
        没有找到错误的原因和位置
理解缺陷及其症状
构思和假设缺陷原因
运行数据和调试代码
定位和修复缺陷
回归测试
找到了错误位置并修复了缺陷
    - 方法
      - 蛮干法：若干断点输出变量值判断
      - 回溯法：from出错点回溯人工查找
      - 排除法：根据缺陷信息归纳演绎出一组缺陷原因&位置，逐个证明or反驳
  - BASIS
    - 
      程序代码等
运行代码
实际结果
结果评价
软件缺陷
测试数据
预期结果
测试用例
    - 步骤：明确测试对象(项目\代码段\方法)->设计测试用例->运行->分析
    - 
      开始
制订测试计划
根据设计模型/文档设计测试用例
实施软件活动
单元测试
集成测试
确认测试
有无缺陷?
无
结束
有
程序调试
缺陷修复
回归测试

实现

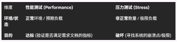
BLAIR'S NOTE

- chapter 13 软件测试
  - ❗ 过程&策略
    - 单元测试：基本模块单元，如过程、函数、方法、类
    - 集成测试：对接口进行测试.
      - 测试方法
        - 非渐增式集成：把所有模块堆在一起，一次性测试
        - 渐增式集成
          - 自顶向下：从主控模块main开始一层一层向下测试(DFS\BFS)
            - 通过桩程序辅助：因为上层要调用下层，下层还没写好，所以要写个假的（Stub）来占位，通常只返回固定数据。
          - 自底向上：从原子模块开始，一层层往上测。
            - 通过驱动程序辅助：因为底层模块是一堆函数，自己动不了，需要写个主程序（Driver）来调用它并输入数据。
      - 回归测试：就是"回头看"，确保修一个Bug没生出俩新Bug。
    - 确认测试：对软件功能和性能进行测试.



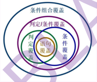
维度	α 测试 (Alpha Testing)	β 测试 (Beta Testing)
谁来测?	**内部人员** (开发公司的员工)	**外部典型用户** (真正的客户)
在哪测?	**开发方现场** (受控环境)	**用户自己的场所** (不受控环境)
测什么?	模拟用户行为，找Bug	实际日常使用，提意见/反馈

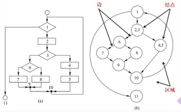
    - 系统测试
      - 恢复测试：故意破坏系统，测试恢复能力
      - 安全测试：测试安全度（攻击数据库）



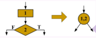
维度	**性能测试 (Performance)**	**压力测试 (Stress)**
环境/状态	**正常**环境 / 预期负载	**非正常**数量 / 极限负载
目的	**达标** (验证是否满足需求文档的指标)	**破坏** (寻找系统的崩溃点/极限)

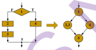
  - 技术
    - 白盒测试：结构测试
      - 覆盖原则
        - 语句覆盖　选择尽可能多的测试数据，使得每个语句都被执行
        - 分支覆盖（判断覆盖）　语句覆盖+每个判定的all可能结果都被执行出
        - 条件覆盖　语句覆盖+每个判定表达式的每个条件取到all可能值
        - ❗ 判断/条件覆盖　判断覆盖+条件覆盖
        - 条件组合覆盖　每个判断表达式中all条件组合都出现

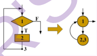

条件组合覆盖
判定/条件覆盖
判定覆盖
语句覆盖
条件覆盖


1
2
3
4
5
6
7
8
9
10
11
(a)
边
结点
1
2,3
4,5
6
7
8
9
10
11
区域
(b)

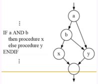
        - ❗ 基本路径覆盖
          - 画出流图
            - 映射方法




F
1
T
2
3
4
5
1
2,3
4
5

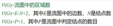


            - 复杂条件分解成简单条件



```
⋮
IF a AND b
   then procedure x
   else procedure y
ENDIF
⋮
```

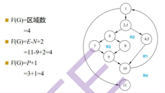
          - 计算环形复杂度



$V(G)$=流图中的区域数

$V(G)=E-N+2$，其中E是流图中的边数，N是结点数

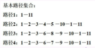
$V(G)=P+1$，其中P是流图中判定结点的数目



- $V(G)$=区域数
  - =4
- $V(G)=E-N+2$
  - =11-9+2=4
- $V(G)=P+1$
  - =3+1=4

1
2,3
4,5
6
7
8
9
10
11
R1
R2
R3
R4

          - 根据复杂度确定基本路径的集合



基本路径集合：

路径1：1—11

路径2：1—2—3—4—5 —10—1—11

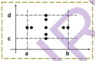
路径3：1—2—3—6—8 —9 —10—1—11

路径4：1—2—3—6—7 —9 —10—1—11

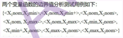
          - 根据集合导出测试用例

一个等价类中的all输入数据作用等效

- 黑盒测试：功能测试
  - 划分等价类
    - 有效等价类：符合规定的\合理的输入
      无效等价类：不符合要求的\错误的输入
      - 设计测试用例(尽可能少)尽可能覆盖有效等价类
        为每一个无效等价类设计一个测试用例
  - ❗ 分析边界值
    - 边界条件处理容易出错，须格外考虑

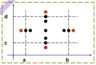


    - 边界值分析

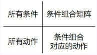


两个变量函数的边界值分析测试用例如下：
{<X_1nom,X_2min>,<X_1nom,X_2min+>,<X_1nom,X_2nom>,
<X_1nom, X_2max->,<X_1nom,X_2max>,<X_1min,X_2nom>,
<X_1min+,X_2nom>,<X_1max-,X_2nom>,<X_1max,X_2nom>}

      - 保留其中一个变量（norm），改变另一个
    - 健壮性边界值分析

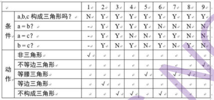


  - ❗ 决策表法



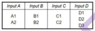
所有条件	条件组合矩阵
所有动作	条件组合 对应的动作

    - e.g.



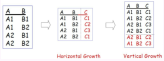
		1	2	3	4	5	6	7	8	9
条件	a,b,c 构成三角形吗？	N	Y	Y	Y	Y	Y	Y	Y	Y
	a = b?	-	Y	Y	Y	Y	N	N	N	N
	a = c?	-	Y	Y	N	N	Y	Y	N	N
	b = c?	-	Y	N	Y	N	Y	N	Y	N
动作	非三角形	√								
	不等边三角形									√
	等腰三角形					√		√	√	
	等边三角形		√							
	不构成三角形			√	√		√			

  - 组合测试
    - 考虑到多个因素间相互作用，会引起交互故障
    - e.g.



Input A	*Input B*	*Input C*	*Input D*
A1 A2	B1 B2	C1 C2	D1 D2 D3

      - all可能组合的组合数 **2*2*2*3=24**
      - pair-wise testing：保证任意两个参数的所有取值组合至少出现一次。
      - T-wise (t=3, 4...)：保证任意 t 个参数 的所有取值组合至少出现一次。
    - IPO 策略 (In-Parameter-Order)
      - 用来生成上述组合测试用例的步骤



A	B
A1	B1
A1	B2
A2	B1
A2	B2

A	B	C
A1	B1	C1
A1	B2	C2
A2	B1	C3
A2	B2	C1

Horizontal Growth

A	B	C
A1	B1	C1
A1	B2	C2
A2	B1	C3
A2	B2	C1
A2	B1	C2
A1	B2	C3

Vertical Growth

  - 模糊测试
    - 软件安全漏洞挖掘技术
    - 暴力输入非预期的随机数据
    - 目的：发现崩溃、异常、安全漏洞